

GUÍA PRÁCTICA DE LABORATORIO

EA1 · FULLSTACK CLOUD NATIVE EN AWS

Angular · Amazon Cognito · API Gateway · EC2 · Spring Boot

OAuth 2.0 / OIDC · Authorization Code + PKCE · JWT · Claims · Scopes · CORS · Interceptor · Spring Security · Defense in Depth

DSY1107 · Desarrollo Cloud Native 1
Prof. Aldo Pizarro · Septiembre 2026

PROPÓSITO

Construir de forma incremental una solución FullStack en AWS, integrando frontend, identidad, API Manager y backend. La guía sigue la misma lógica de la primera práctica: hacer funcionar cada capa, probarla y recién después agregar seguridad. Los nombres, endpoints y datos son ejemplos didácticos; cada proyecto debe definir sus propios recursos y decisiones.

REGLA DE ORO DEL LABORATORIO

No diagnostiques Angular, Cognito, API Gateway y Spring Boot al mismo tiempo. En cada checkpoint valida una sola capa y conserva evidencia de la prueba.

1. MAPA COMPLETO DE LA EXPERIENCIA DE APRENDIZAJE 1

Arquitectura objetivo de referencia:

Usuario → Angular → Amazon Cognito → Access Token JWT
↓
Angular → API Gateway → Cognito Authorizer → EC2 / Spring Boot → Datos

Al finalizar la experiencia, el estudiante debería poder explicar y demostrar:

- Qué función cumple un API Manager y por qué el frontend no debería acoplarse directamente al backend.
- Cómo OAuth 2.0 y OIDC participan en el inicio de sesión y emisión de tokens.
- Diferencia entre ID Token y Access Token.
- Cómo un custom scope representa un permiso de acceso a una API.
- Cómo Angular obtiene sesión, consume la API y automatiza el Bearer token.
- Cómo API Gateway aplica CORS, authorizer y scopes.
- Cómo Spring Boot valida nuevamente el JWT como Resource Server.
- Por qué validar en más de una capa corresponde a Defense in Depth.
- Cómo versionar rutas y diagnosticar 200, 401, 403, CORS y errores de integración.

CHECKPOINT · Arquitectura

Antes de crear recursos, el estudiante puede dibujar el flujo E2E e identificar frontend, IDaaS, API Manager y backend.

2. PREPARACIÓN DEL LABORATORIO

Herramientas sugeridas: AWS Academy/Learner Lab, Java 17 o 21, Maven Wrapper, Node.js, Angular CLI, VS Code/IDE, Postman y navegador con DevTools.

Comprueba localmente:

```
java -version
node -v
npm -v
ng version
git --version
```

ATENCIÓN · Valores del proyecto

Durante la guía usa variables como <REGION>, <USER_POOL_ID>, <APP_CLIENT_ID>, <API_ID>, <STAGE>, <EC2_PUBLIC_IP> y <CUSTOM_SCOPE>. No copies identificadores de otro equipo.

3. BACKEND SPRING BOOT: CREAR PRIMERO UNA API SIMPLE

3.1 Crear el proyecto

1. Spring Initializr → Maven → Java.
2. Selecciona una versión estable de Spring Boot compatible con Java 17/21.
3. Dependencia inicial: Spring Web.
4. Group de ejemplo: cl.duoc.
5. Artifact de ejemplo: pedidos360-backend.

Estructura mínima:

```
src/main/java/cl/duoc/pedidos360_backend/  
    Pedidos360BackendApplication.java  
    controller/  
        Pedidos360Controller.java  
src/main/resources/  
    application.properties
```

3.2 Crear el endpoint

```
package cl.duoc.pedidos360_backend.controller;  
  
import java.util.List;  
import java.util.Map;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
@RestController  
@RequestMapping("/api/pedidos")  
public class Pedidos360Controller {  
  
    @GetMapping  
    public List<Map<String, Object>> obtenerPedidos() {  
        return List.of(  
            Map.of("id", 1, "producto", "Notebook", "estado", "EN_PREPARACION"),  
            Map.of("id", 2, "producto", "Telefono", "estado", "DISPONIBLE")  
        );  
    }  
}
```

3.3 Ejecutar y probar localmente

```
./mvnw spring-boot:run
```

Prueba:

GET <http://localhost:8080/api/pedidos>

CHECKPOINT · Backend local

El endpoint responde HTTP 200 con JSON. Si falla aquí, todavía NO avances a EC2 ni API Gateway.

4. DESPLEGAR SPRING BOOT EN EC2

4.1 Crear instancia

1. EC2 → Launch instance.
2. Usa una AMI disponible en el laboratorio (por ejemplo Amazon Linux).
3. Asocia key pair si el laboratorio lo requiere.
4. Asocia a la instancia un IAM Role con permisos de lectura sobre el bucket S3 utilizado para distribuir el artefacto del backend.
5. Para la integración pública temporal del laboratorio, habilita el puerto del backend (por ejemplo 8888).

ATENCIÓN · Seguridad

Abrir 8888 a 0.0.0.0/0 puede servir para el laboratorio, pero NO representa una arquitectura final recomendada.

4.2 Preparar S3 e IAM Role para entregar el artefacto

1. Crea o reutiliza un bucket S3 del laboratorio para almacenar el archivo JAR generado por Spring Boot.
2. Genera el JAR con Maven y súbelo al bucket S3. El bucket no necesita ser público.
3. Crea o modifica el IAM Role de la instancia EC2 para permitir lectura sobre ese bucket S3.
4. En EC2 → Instances → selecciona la instancia → Actions → Security → Modify IAM role y asocia el role correspondiente.
5. Verifica desde la instancia que AWS reconoce el role asociado:

```
aws sts get-caller-identity
```

Comprueba que la instancia puede acceder al bucket autorizado:

```
aws s3 ls s3://<NOMBRE_BUCKET>/
```

CHECKPOINT: EC2 puede leer el artefacto desde S3 mediante su IAM Role, sin Access Key ni Secret Key configuradas en la instancia.

4.3 Instalar Java

```
sudo dnf install java-17-amazon-corretto -y
java -version
```

4.4 Cambiar puerto de la aplicación

```
server.port=8888
```

4.5 Empaquetar, subir a S3 y ejecutar en EC2

En este laboratorio distribuiremos el JAR mediante Amazon S3. EC2 obtiene el archivo utilizando el IAM Role asociado a la instancia. No utilizaremos SCP/SSH para copiar el artefacto ni almacenaremos credenciales AWS estáticas en el servidor.

```
# En el equipo donde desarrollas el backend
./mvnw clean package
```

```
# Subir el artefacto a S3
aws s3 cp target/<nombre-del-jar>.jar s3://<NOMBRE_BUCKET>/<nombre-del-jar>.jar
```

```
# En EC2: descargar el artefacto usando el IAM Role
aws s3 cp s3://<NOMBRE_BUCKET>/<nombre-del-jar>.jar ./<nombre-del-jar>.jar
```

```
# Ejecutar Spring Boot
java -jar <nombre-del-jar>.jar
```

Para dejarlo ejecutándose:

```
java -jar <nombre-del-jar>.jar
```

Prueba desde el navegador/Postman:

```
GET http://<EC2_PUBLIC_IP>:8888/api/pedidos
```

CHECKPOINT · Backend en EC2

La IP pública y el puerto responden HTTP 200. Registra la URL; será la integración de API Gateway.

5. API GATEWAY: CREAR EL PUNTO DE ENTRADA

5.1 Crear REST API

1. AWS Console → API Gateway → Create API.
2. Selecciona REST API → Build.
3. Crea una API con nombre identificable.
4. Resources → Create resource.
5. Crea un recurso de ejemplo /api y debajo /pedidos, o una ruta equivalente definida por tu proyecto.
6. Sobre /api/pedidos crea método GET.
7. Integration type: HTTP.
8. Endpoint URL: http://<EC2_PUBLIC_IP>:8888/api/pedidos.
9. Si está disponible y corresponde al diseño, utiliza HTTP proxy integration.

5.2 Probar integración antes de seguridad

Usa Test en el método GET. Luego despliega:

1. Deploy API.
2. Crea/selecciona stage test.
3. Obtén Invoke URL.
4. Prueba desde Postman.

GET `https://<API_ID>.execute-api.<REGION>.amazonaws.com/test/api/pedidos`

CHECKPOINT · API Gateway sin seguridad

API Gateway responde 200 y entrega el mismo JSON del backend. Recién ahora agrega identidad.

6. CORS: PERMITIR AL FRONTEND LLAMAR LA API

Postman no aplica la política CORS del navegador. Angular sí. Por eso una API puede funcionar en Postman y fallar en Chrome.

1. Resources → selecciona /api/pedidos.
2. Enable CORS.
3. Allowed Origin de laboratorio: `http://localhost:4200`.
4. Allowed Methods: GET, OPTIONS.
5. Allowed Headers: Content-Type, Authorization.
6. Confirma la creación/configuración de OPTIONS.
7. Deploy API nuevamente al stage test.

Access-Control-Allow-Origin: `http://localhost:4200`

Access-Control-Allow-Headers: `Content-Type,Authorization`

Access-Control-Allow-Methods: `GET,OPTIONS`

ATENCIÓN · CORS en errores

Si OPTIONS=200 pero GET=401/403 aparece como status 0 o “blocked by CORS”, configura también Gateway Responses DEFAULT_4XX y DEFAULT_5XX con Access-Control-Allow-Origin. Así el navegador podrá mostrar el error real.

CHECKPOINT · CORS

En DevTools → Network, OPTIONS responde 200. El navegador puede realizar el GET sin bloquear la respuesta por política CORS.

7. AMAZON COGNITO: CREAR EL IDaaS/CIAM

7.1 Crear User Pool

1. Amazon Cognito → User pools → Create user pool.
2. Define el mecanismo de inicio de sesión requerido por el laboratorio.
3. Configura atributos y políticas de contraseña.
4. Crea el User Pool.
5. Registra el User Pool ID y la región.

7.2 Crear App Client

1. Applications → App clients → Create app client.
2. Para SPA usa un cliente público; no dependas de un client secret en Angular.
3. Registra el App Client ID.
4. Habilita el flujo de Authorization Code para el login administrado.

7.3 Dominio y callback

1. Branding → Domain → crea/identifica el dominio administrado.
2. App client → Login pages → Edit.
3. Callback URL para Angular: `http://localhost:4200`.
4. Sign-out URL: `http://localhost:4200`.
5. Para Postman puedes agregar `https://oauth.pstmn.io/v1/callback`.
6. Habilita scopes OIDC: openid, email, profile según necesidad.

CHECKPOINT · Cognito base

Tienes User Pool ID, App Client ID, dominio, callback y Authorization Code habilitado.

8. OAUTH 2.0, OIDC Y AUTHORIZATION CODE + PKCE

OAuth 2.0 se enfoca en autorización; OIDC agrega identidad sobre OAuth 2.0. En una SPA moderna, Authorization Code + PKCE evita exponer un client secret en el navegador.

Elemento	Rol en el laboratorio
Authorization endpoint	Inicia autenticación y consentimiento.
Token endpoint	Intercambia el authorization code por tokens.
PKCE	Vincula la solicitud de autorización con el intercambio del code.
ID Token	Información de identidad para el cliente.
Access Token	Token que se presenta para acceder a la API.

ATENCIÓN · Token correcto
Para autorizar la API usa Access Token. No reemplaces el Access Token por el ID Token solo porque ambos sean JWT.

9. RESOURCE SERVER Y CUSTOM SCOPES EN COGNITO

- Cognito → User Pool → Applications/Resource servers.
- Create resource server.
- Define un Identifier propio, por ejemplo pedidos360-api.
- Crea un scope, por ejemplo pedidos-api-read.
- El scope completo será pedidos360-api/pedidos-api-read.
- Vuelve al App Client → Login pages.
- Habilita el custom scope para el cliente.

<RESOURCE_SERVER_IDENTIFIER>/<SCOPE_NAME>
Ejemplo: pedidos360-api/pedidos-api-read

CHECKPOINT · Scope
El App Client está autorizado para solicitar el custom scope. No basta con crear el scope: debe estar habilitado en el cliente.

10. PROBAR COGNITO CON POSTMAN ANTES DE ANGULAR

Configura OAuth 2.0 en Postman:

Campo	Valor
Grant Type	Authorization Code (With PKCE)
Callback URL	https://oauth.pstmn.io/v1/callback
Auth URL	https://<DOMINIO>/oauth2/authorize
Access Token URL	https://<DOMINIO>/oauth2/token
Client ID	<APP_CLIENT_ID>
Client Secret	Vacío para cliente público
Code Challenge Method	SHA-256
Scope	openid email profile <CUSTOM_SCOPE>

- Get New Access Token.
- Completa login.
- Usa el Access Token obtenido.
- Decodifica el payload para observar claims.

Claims esperables conceptualmente:

```
token_use: "access"
scope: "openid ... <CUSTOM_SCOPE>"
client_id: "<APP_CLIENT_ID>"
iss: "https://cognito-idp.<REGION>.amazonaws.com/<USER_POOL_ID>"
```

CHECKPOINT · OAuth
Postman obtiene Access Token y el payload contiene token_use=access y el custom scope solicitado.

11. COGNITO AUTHORIZER EN API GATEWAY

11.1 Crear authorizer

- 1. API Gateway → REST API → Authorizers.
- 2. Create authorizer.
- 3. Type: Cognito.
- 4. Selecciona el User Pool.
- 5. Token source: Authorization.
- 6. Deja vacío Token validation regex salvo que exista una necesidad explícita.
- 7. Guarda.

11.2 Asociar al método

- 1. Resources → GET /api/pedidos.
- 2. Method Request → Edit.
- 3. Authorization → selecciona el Cognito Authorizer.
- 4. Authorization scopes → agrega exactamente <CUSTOM_SCOPE>.
- 5. Guarda.
- 6. Deploy API nuevamente al stage test.

ATENCIÓN · Deploy

En REST API los cambios no llegan al stage hasta volver a desplegar.

12. MATRIZ DE PRUEBAS DE SEGURIDAD EN API GATEWAY

Prueba	Header Authorization	Resultado esperado
Sin token	No	401 / rechazo de autenticación
Token inventado	Bearer abc	401
Access Token válido sin permiso requerido	Bearer <JWT>	403 o rechazo de autorización según configuración
Access Token válido con scope	Bearer <JWT>	200 + JSON

En Postman:

Authorization: Bearer <ACCESS_TOKEN>

CHECKPOINT · API protegida

Sin token falla; con Access Token válido y scope correcto responde 200.

13. CREAR EL FRONTEND ANGULAR

13.1 Crear proyecto

```
npm install -g @angular/cli
ng new pedidos360-front
cd pedidos360-front
npm start
```

Comprueba http://localhost:4200.

13.2 Instalar Amplify

```
npm install aws-amplify
```

CHECKPOINT · Angular base

La aplicación levanta en localhost:4200 antes de integrar Cognito.

14. CONFIGURAR AWS AMPLIFY EN ANGULAR

En src/main.ts configura Amplify antes de bootstrapApplication. Sustituye los placeholders por TUS valores.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';
import { Amplify } from 'aws-amplify';

Amplify.configure({
  Auth: {
    Cognito: {
      userPoolId: '<USER_POOL_ID>',
      userPoolClientId: '<APP_CLIENT_ID>',
      loginWith: {
        oauth: {
          domain: '<COGNITO_DOMAIN>',
          scopes: [
            'email',
            'openid',
            'profile',
            '<CUSTOM_SCOPE>'
          ],
          redirectSignIn: ['http://localhost:4200'],
          redirectSignOut: ['http://localhost:4200'],
          responseType: 'code'
        }
      }
    }
  }
});
bootstrapApplication(App, appConfig)
  .catch(err => console.error(err));
```

ATENCIÓN · Coincidencia exacta

Callback URL, Sign-out URL, dominio, User Pool ID, Client ID y scopes deben coincidir con Cognito. Un carácter distinto puede romper el flujo.

15. LOGIN, LOGOUT Y SESIÓN EN ANGULAR

En el componente principal importa las funciones de Amplify Auth:

```
import {
  signInWithRedirect,
  signOut,
  fetchAuthSession,
  getCurrentUser
} from 'aws-amplify/auth';
```

Métodos didácticos:

```
usuario = "";
token = "";
autenticado = false;
async login() {
  await signInWithRedirect();
}
async logout() {
  await signOut();
}
async verSesion() {
  try {
    const user = await getCurrentUser();
    const session = await fetchAuthSession();
    this.usuario = user.username;
    this.token = session.tokens?.accessToken?.toString() ?? "";
    this.autenticado = true;
    console.log('Usuario:', user);
    console.log('Access Token:', this.token);
  } catch (error) {
    console.log('No existe sesión', error);
    this.autenticado = false;
  }
}
```

En la vista crea botones para iniciar sesión, ver sesión y cerrar sesión. Mostrar el token sirve solo con fines pedagógicos; no es una recomendación de UX productiva.

CHECKPOINT · Login Angular

Login redirige a Cognito y regresa a localhost:4200. Ver sesión obtiene un Access Token.

16. PRIMER CONSUMO DE LA API: TOKEN MANUAL

Antes de automatizar, conviene demostrar explícitamente qué hace el cliente.

```
pedidos: any[] = [];
cargandoPedidos = false;
errorPedidos = '';
async consultarPedidos() {
  this.cargandoPedidos = true;
  this.errorPedidos = '';
  try {
    const session = await fetchAuthSession();
    const token = session.tokens?.accessToken?.toString();

    if (!token) {
      this.errorPedidos = 'No existe una sesión autenticada.';
      return;
    }
    const response = await fetch('<URL_API_GATEWAY>', {
      method: 'GET',
      headers: {
        Authorization: `Bearer ${token}`
      }
    });
    if (!response.ok) {
      this.errorPedidos = `HTTP ${response.status}`;
      return;
    }
    this.pedidos = await response.json();
  } catch (error) {
    console.error(error);
    this.errorPedidos = 'No fue posible consultar la API.';
  } finally {
    this.cargandoPedidos = false;
  }
}
```

CHECKPOINT · E2E manual

Usuario autenticado → Angular obtiene Access Token → GET con Bearer → API Gateway valida → backend responde JSON.

17. EVOLUCIONAR ANGULAR: HTTPCLIENT + SERVICE

17.1 Crear servicio

ng generate service pedidos

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
@Injectable({ providedIn: 'root' })
export class PedidosService {
  private readonly url = '<URL_API_GATEWAY>';

  constructor(private http: HttpClient) {}

  obtenerPedidos(): Observable<any[]> {
    return this.http.get<any[]>(this.url);
  }
}
```

La responsabilidad de conocer la URL y realizar la llamada deja de estar mezclada con la vista.

18. INTERCEPTOR: AGREGAR EL JWT AUTOMÁTICAMENTE

18.1 Crear auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { fetchAuthSession } from 'aws-amplify/auth';
import { from, switchMap } from 'rxjs';
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  return from(fetchAuthSession()).pipe(
    switchMap(session => {
      const token = session.tokens?.accessToken?.toString();
      if (!token) {
        return next(req);
      }
      const requestConToken = req.clone({
        setHeaders: {
          Authorization: `Bearer ${token}`
        }
      });
      return next(requestConToken);
    })
  );
};
```

18.2 Registrar HttpClient e interceptor

Este punto es crítico: provideHttpClient(...) debe quedar DENTRO de providers.

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './auth.interceptor';
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(
      withInterceptors([authInterceptor])
    )
  ]
};
```

ATENCIÓN · Error frecuente

Si provideHttpClient(withInterceptors(...)) queda después de cerrar appConfig, el archivo puede parecer correcto visualmente pero el interceptor NO queda registrado.

18.3 Consumir desde el componente

```
constructor(private pedidosService: PedidosService) {}
consultarPedidos() {
  this.cargandoPedidos = true;
  this.pedidosService.obtenerPedidos().subscribe({
    next: data => {
      this.pedidos = data;
      this.cargandoPedidos = false;
    },
    error: error => {
      console.error(error);
      this.errorPedidos = 'No fue posible consultar pedidos';
      this.cargandoPedidos = false;
    }
  });
}
```

CHECKPOINT · Interceptor

DevTools → Network → GET → Request Headers contiene Authorization: Bearer eyJ... sin agregarlo manualmente en el componente.

19. GUARDS Y RUTAS PROTEGIDAS

El interceptor protege llamadas HTTP; un Guard controla navegación. Son responsabilidades distintas.

Flujo conceptual de un Guard:

Usuario intenta /pedidos → Guard consulta sesión → sesión válida: permite → sin sesión: inicia/redirige a login

ATENCIÓN · No confundir

Ocultar una ruta en Angular NO protege por sí solo la API. La autorización real debe seguir ocurriendo en API Gateway y/o backend.

Si tu aplicación todavía no usa rutas, comprende el patrón antes de crear estructura artificial solo para “tener un Guard”. Cuando existan rutas privadas, aplícalo.

20. SPRING SECURITY: BACKEND COMO RESOURCE SERVER

20.1 Agregar dependencias

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

20.2 Configurar issuer

spring.security.oauth2.resourceserver.jwt.issuer-uri=https://cognito-idp.<REGION>.amazonaws.com/<USER_POOL_ID>

20.3 SecurityConfig

```
package cl.duoc.pedidos360_backend.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
public class SecurityConfig {
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/pedidos").authenticated()
                .anyRequest().permitAll()
            )
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {}));
        return http.build();
    }
}
```

20.4 Reconstruir y desplegar

```
./mvnw clean package
java -jar target/<jar>.jar
```

CHECKPOINT · Resource Server

Solicitud directa al backend SIN JWT deja de entregar datos y responde 401. Con un JWT válido puede ser autenticada por Spring Security.

21. AUTORIZACIÓN POR SCOPES EN SPRING SECURITY

Spring Security puede convertir scopes del JWT en authorities con prefijo SCOPE_. Conceptualmente:

```
scope: pedidos360-api/pedidos-api-read
→ authority: SCOPE_pedidos360-api/pedidos-api-read
```

Ejemplo de regla más estricta:

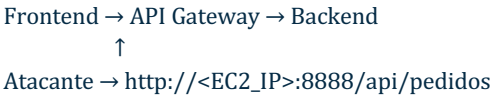
```
.requestMatchers("/api/pedidos")
.hasAuthority("SCOPE_<CUSTOM_SCOPE>")
```

ATENCIÓN · Validación de Access Token Cognito

Además de firma, expiración e issuer, una solución estricta puede validar que el token sea de uso access y que corresponda al cliente esperado. No asumas que un Access Token de Cognito siempre contiene el mismo claim aud que un ID Token; Cognito utiliza client_id en el Access Token.

22. DEFENSE IN DEPTH Y LA “PUERTA TRASERA”

Antes de Spring Security, esta ruta puede existir:



Si el backend estaba abierto y sin validación, llamar directamente EC2 evitaba el Authorizer de API Gateway. Al convertir Spring Boot en Resource Server, una llamada directa sin JWT también es rechazada.

ATENCIÓN · Matiz importante

Si API Gateway usa una integración HTTP pública hacia EC2, cerrar simplemente el puerto 8888 en el Security Group puede romper también API Gateway. La protección lógica con Spring Security no equivale a una red privada. Para restringir el ingreso a nivel de red se requiere evolucionar la arquitectura hacia una integración privada apropiada (por ejemplo VPC Link y componentes de balanceo/privacidad compatibles con el tipo de API).

CHECKPOINT · Defense in Depth

API Gateway valida el token y el backend vuelve a validarlo. El estudiante puede explicar por qué existen dos controles.

23. VERSIONAMIENTO DE APIS

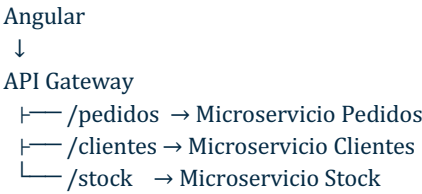
Evita romper consumidores al cambiar contratos. Una estrategia visible es versionar la ruta:

```
GET /v1/pedidos
GET /v2/pedidos
```

API Gateway puede enrutar versiones hacia integraciones distintas o hacia diferentes endpoints del mismo backend. El versionamiento debe responder a una decisión de compatibilidad, no solo a cambiar el nombre de la URL.

24. ARQUITECTURA CON VARIOS MICROSERVICIOS

Si el proyecto crece, API Gateway puede concentrar múltiples rutas:



Cada servicio mantiene su responsabilidad. No conviertas API Gateway en lógica de negocio ni hagas que el frontend conozca direcciones internas de cada microservicio.

25. BASE DE DATOS CLOUD Y CAPAS

Cuando el backend incorpore persistencia, mantén la separación de responsabilidades:

```
Controller → Service → Repository → Base de datos
```

El frontend nunca debería conectarse directamente a la base de datos. Las credenciales tampoco deben quedar en Angular ni versionadas en Git.

26. GITHUB Y CONFIGURACIÓN SEGURA

Antes de publicar:

- Incluye .gitignore apropiado para Java/Node.
- No subas target/, node_modules/, archivos del IDE ni secretos.
- Separa configuración por ambiente cuando corresponda.
- Documenta cómo levantar frontend y backend.
- Verifica que el repositorio compile desde una copia limpia.

ATENCIÓN · Qué sí puede verse en una SPA

El App Client ID de un cliente público no se trata como un client secret. Lo que nunca debes incrustar en Angular es una credencial secreta que otorgue privilegios por sí misma.

27. TROUBLESHOOTING POR CAPAS

Síntoma	Capa probable	Qué revisar primero
Angular queda en “Cargando...”	Frontend/red	DevTools Network; finally/error; solicitud Pending.
CORS policy / status 0	API Gateway CORS	OPTIONS, Allow-Origin, Allow-Headers, Gateway Responses 4XX/5XX.
OPTIONS 200, GET 401	Autenticación	Authorization Bearer, Access Token, authorizer.
GET 403	Autorización	Custom scope solicitado, emitido y exigido.
502/504	Integración	EC2 activo, puerto, URL de integración, IP pública actual.
Postman funciona, navegador no	CORS	Origin y preflight.
Token existe pero interceptor no lo envía	Angular config	provideHttpClient(withInterceptors(...)) dentro de providers.
API Gateway funciona pero Spring responde 401	Resource Server	issuer, JWT, Authorization forwarding.
EC2 dejó de responder tras reinicio	Infraestructura	IP pública puede haber cambiado; proceso Java y SG.

28. DIAGNÓSTICO EN DEVTOOLS

Chrome/Edge → DevTools → Network → Fetch/XHR. Observa por separado OPTIONS y GET.

En GET revisa Request Headers:

Authorization: Bearer eyJ...

Interpretación rápida:

- OPTIONS 200 + GET 200 → flujo correcto
- OPTIONS bloqueado → CORS/preflight
- GET 401 → autenticación/JWT/authorizer
- GET 403 → autorización/scope
- GET 5xx → integración/backend

29. PRUEBA E2E FINAL

Realiza la demostración en este orden para poder aislar errores:

1. Backend local responde.
2. Backend EC2 responde.
3. API Gateway sin seguridad responde.
4. Cognito emite Access Token.
5. API Gateway protegido rechaza sin token.
6. Postman con token y scope responde.
7. Angular inicia sesión.
8. Angular obtiene sesión.
9. Angular llama con HttpClient.
10. Interceptor agrega Bearer.
11. API Gateway valida JWT y scope.
12. Spring Security vuelve a validar JWT.
13. La UI muestra los datos.

CHECKPOINT · E2E

Usuario → Angular → Cognito → Access Token → API Gateway → Cognito Authorizer → Spring Security → Controller → JSON → Angular.

30. CHECKLIST DE AUTOEVALUACIÓN EA1

- [] 01. Puedo explicar API Manager, IDaaS, OAuth 2.0 y OIDC con mis palabras.
- [] 02. Mi backend compila y puede probarse de forma independiente.
- [] 03. Mi backend está desplegado y sé revisar sus logs.
- [] 04. API Gateway es el punto de entrada del frontend.
- [] 05. Mis rutas y métodos representan recursos coherentes.
- [] 06. CORS permite el origen del frontend y Authorization.
- [] 07. Tengo User Pool, App Client y dominio configurados.
- [] 08. Authorization Code + PKCE funciona.
- [] 09. Conozco la diferencia entre ID Token y Access Token.
- [] 10. El Access Token contiene el custom scope requerido.
- [] 11. API Gateway usa Cognito Authorizer.
- [] 12. Sin token obtengo rechazo; con token válido obtengo acceso.
- [] 13. Angular usa Amplify para login/logout/sesión.
- [] 14. Las llamadas de negocio están separadas en un Service.
- [] 15. HttpClient tiene registrado el interceptor.
- [] 16. El interceptor agrega Authorization: Bearer automáticamente.
- [] 17. Comprendo cuándo usar un Guard.
- [] 18. Spring Boot funciona como OAuth2 Resource Server.
- [] 19. Puedo explicar Defense in Depth y el bypass directo a EC2.
- [] 20. Comprendo cómo evolucionar a versionamiento y múltiples microservicios.
- [] 21. Mi repositorio no contiene secretos ni artefactos innecesarios.
- [] 22. Puedo diagnosticar 200, 401, 403, CORS y 5xx por capas.

31. RECORRIDO RECOMENDADO PARA CONSTRUIR UN PROYECTO

- 1 Backend local
- 2 Backend EC2
- 3 API Gateway sin seguridad
- 4 CORS
- 5 Cognito User Pool + App Client + dominio
- 6 Resource Server + custom scope
- 7 Postman OAuth2/PKCE
- 8 Cognito Authorizer + scope
- 9 Angular + Amplify
- 10 Login/sesión
- 11 Consumo manual con Bearer
- 12 HttpClient + Service
- 13 Interceptor
- 14 Guard cuando existan rutas privadas
- 15 Spring Security Resource Server
- 16 Scope en backend
- 17 Defense in Depth
- 18 Versionamiento
- 19 E2E + evidencias

CIERRE

La meta no es memorizar clics. La meta es poder justificar cada capa: quién autentica, quién autoriza, quién expone la API, quién valida el token, qué permiso se exige y cómo diagnosticar el flujo cuando una capa falla.